



HACKATHON TALK

SOCIAL IMPACT AI

PYTHON TOOLS

From Hello World to Hello LLM

A Python Developer's Survival Guide

Lessons from a Social Impact AI Hackathon

Audrey Webb

Senior ML Engineer at Autodesk

/dev/color Fellow



Jasmine Omeke

Senior Software Engineer, Data Infrastructure at Airbnb

/dev/color Fellow



3 Days, One Hackathon, Two Solutions

...and creating a nation-wide, durable model for education and AI integration



Student success dashboard with **predictive analytics & AI chatbot**



Programmatic and self-serve AI chatbot for
answering research requests and interpreting
student success data
(i.e., “toxic course combinations”)

AI collapses the gap between expertise & execution



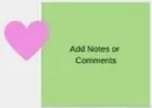
Before We Build: Understand User Needs

Requirements gathering was an important step to help us understand the goals of our institutions and define:

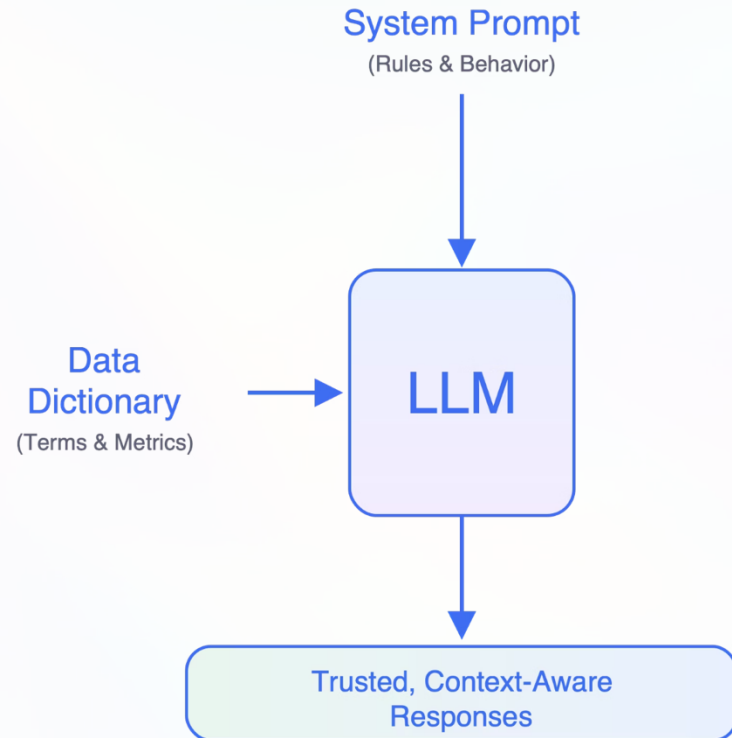
- what kinds of questions the system should answer
- what data it was allowed to access
- what topics were out of scope
- how responses should be phrased

User Journey Map

User Journey Maps give an overview of the customer experience. How do you want your business to reach users?

	STAGE 1	STAGE 2	STAGE 3	STAGE 4	STAGE 5
OBJECTIVES	Write a goal or activity				
NEEDS	Write a need you want to meet				
FEELINGS	Write an emotion you expect the customer to have				
BARRIERS	Write a potential challenge to your objective				

Core Pattern 1: System Prompts + Data Dictionaries



```
INSTRUCTION_LAYER = ""
You are an assistant for higher education staff.
Rules:
- Use only approved metric definitions
- Return aggregate data only (no PII)
""

METRIC_DEFINITIONS = {
    "retention": "students who re-enroll",
    "persistence": "term-to-term continuation",
    "completion": "program completion"
}

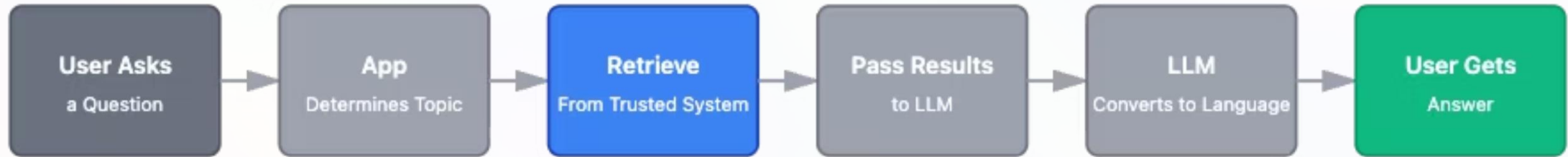
SUMMARIZED_LOG_DATA = {
    ...
}

# Instruction Layer → governs behavior
# Metric Definitions → governs meaning of data
# Log Data → tracks unsaid interactions in UI
# Together → enforce data governance
+ consistent interpretation
```

👍 **Your takeaway:** System prompts + data dictionaries turn language into a controlled interface for data.

Core Pattern 2: Retrieval-Augmented Generation (RAG)

USER EXPERIENCE



✓ **Your takeaway:** RAG connects the LLM to live institutional data systems.

RAG In Practice

```
# Data sources (swappable, secured)
DATA_SOURCES = {
    "sql": "...Azure SQL / AAD auth..."
}

# Retrieval layer
def fetch_metrics(query):
    if "retention" in query:
        return sql("""
            SELECT metric, value, definition
            FROM student_metrics
            WHERE metric = 'retention'
        """)
    return None
```

```
# LLM layer (swappable)
def call_llm(prompt, model="gpt-4o"):
    return llm.chat(model=model, messages=[
        {"role": "system", "content": "Use only provided context."},
        {"role": "user", "content": prompt}
    ])

# App flow
def answer(query, model="gpt-4o"):
    context = fetch_metrics(query)

    prompt = f"{query}\n\nContext:\n{context}"

    response = call_llm(prompt, model=model)

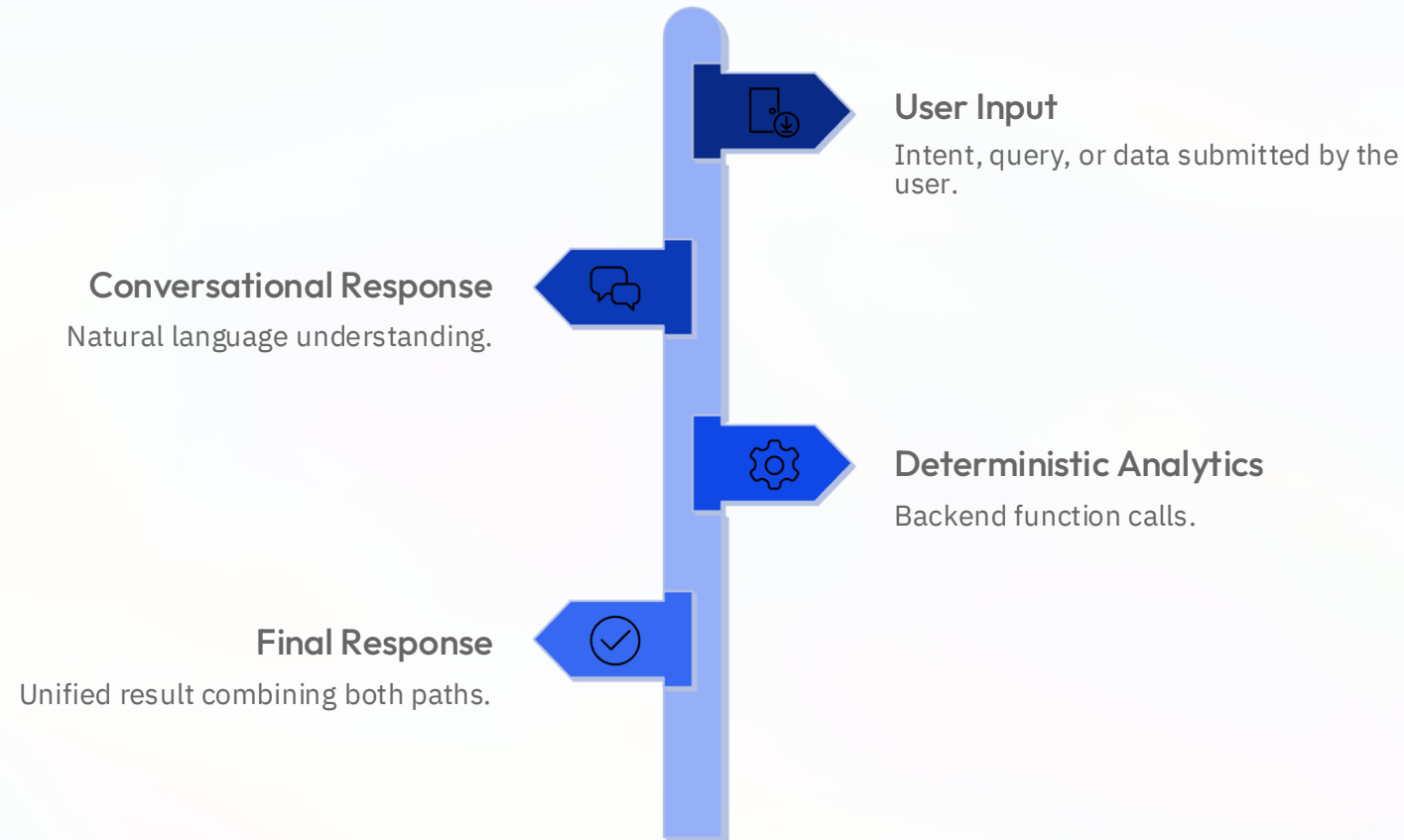
    log = {"query": query, "model": model}

    return response, log
```

✅ **Your takeaway:** We pull approved data from external systems at query time and use it as context for swappable, auditable LLMs.

Core Pattern 3: Function Calling & Controlled Output

Not every problem requires an LLM.



✓ **Your takeaway:** Two paths, one outcome: the LLM handles interpretation, while backend systems handle controlled analytics when precision matters.

Function Calling In Practice

```
from openai import OpenAI

client = OpenAI()

# Predictive model wrapped as an approved function
def get_retention_risk(school_id):

    students = feature_store.get_students(school_id)

    risk_scores = retention_model.predict_proba(students)

    return {
        "school_id": school_id,
        "avg_retention_risk": float(risk_scores.mean())
    }

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_retention_risk",
            "description": "Retrieve student retention risk metrics",
            "parameters": {
                "type": "object",
                "properties": {
                    "school_id": {"type": "string"}
                },
                "required": ["school_id"]
            }
        }
    }
]
```

```
user_question = (
    "Which students are most at risk of not returning next semester?"
)

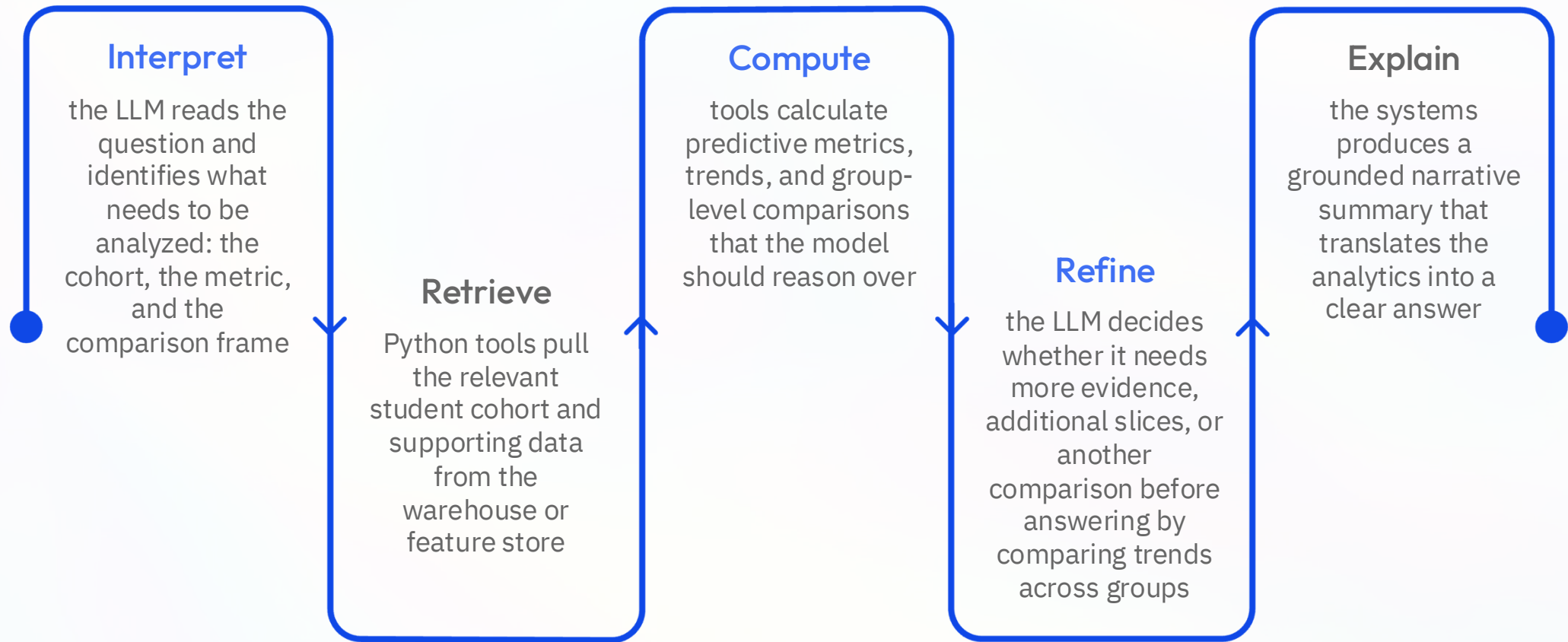
# Step 1: LLM determines what analytics are needed
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "system",
            "content": (
                "Never generate predictions yourself. "
                "Use available analytics functions and explain results."
            )
        },
        {"role": "user", "content": user_question}
    ],
    tools=tools
)

# Step 2: Backend executes predictive model
analytics_result = get_retention_risk(
    school_id="School_123"
)

# Step 3: LLM explains results
final_response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": user_question},
        {
            "role": "tool",
            "tool_call_id": response.choices[0].message.tool_calls[0].id,
            "content": str(analytics_result)
        }
    ]
)

print(final_response.choices[0].message.content)
```

Core Pattern 4: Simple Agent Loops



Agent Loops In Practice

```
# Available tools
tools = [
    get_student_cohort,
    get_retention_risk,
    compare_to_previous_term
]

query =
"""
Which first-year students are most at risk,
and is retention improving or declining?
"""

# Agent loop
cohort = get_student_cohort(
    year="first_year"
)
```

```
risk_scores = get_retention_risk(
    students=cohort
)

trend = compare_to_previous_term(
    current=risk_scores
)

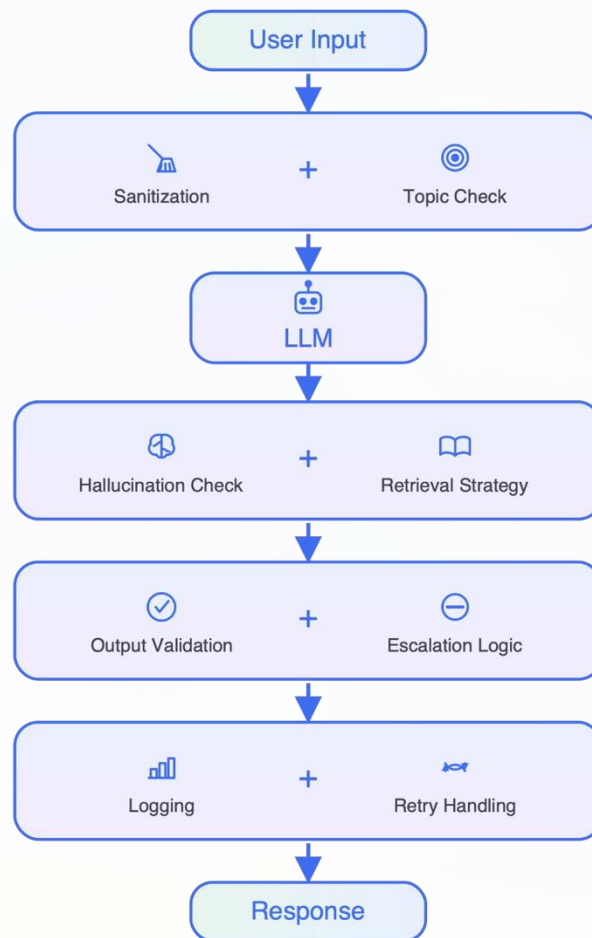
summary = llm.generate(
    f"""
    Cohort: {cohort}
    Risk Scores: {risk_scores}
    Trend Analysis: {trend}

    Summarize findings for university advisors.
    """
)

print(summary)
```

✅ **Your takeaway:** The loop combines intent interpretation, data retrieval, computation, and summarization into one workflow and response.

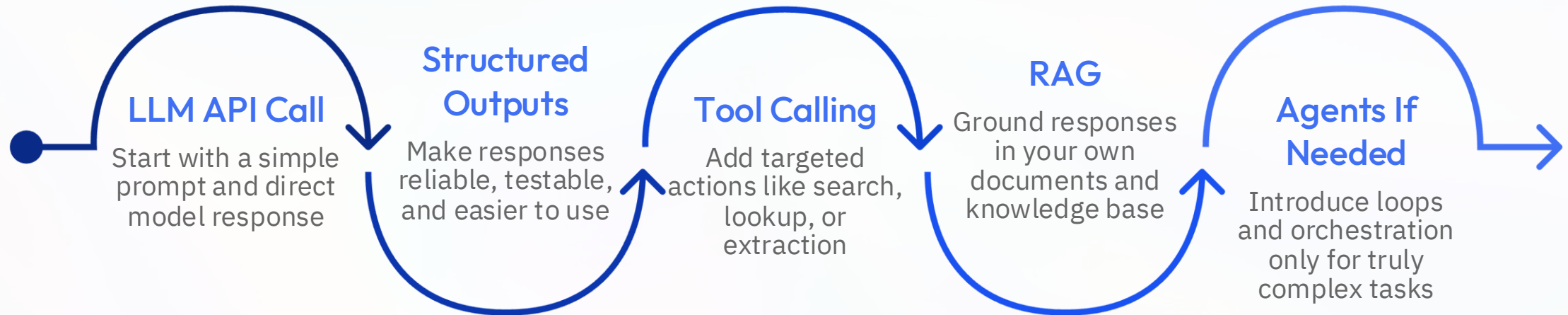
LLM Guardrails We Deployed



✓ **Your takeaway:** The goal is not to rely solely on the model, but to build systems that remain reliable even when the LLM makes mistakes.

Putting It All Together

The path forward is straightforward: start simple, add capability only when it earns its keep, and keep the system understandable as you go.



Build in This Order

1. Start with a simple LLM API call
2. Add structured outputs to make results dependable
3. Add tool calling for focused, useful actions
4. Add RAG when external knowledge matters
5. Introduce agents only when the task genuinely requires them

What Matters Most

- You do not need deep ML expertise to build useful AI systems
- Strong engineering fundamentals matter more than clever prompts alone
- Clear abstractions keep the system maintainable
- Careful system design reduces cost, latency, and failure modes
- Start small, observe everything, and expand with purpose

✓ **Your takeaway:** The path forward is straightforward. Build on solid engineering basics, add capability in layers, and use the simplest abstraction that solves the problem.

Appendix

Here are some of the open-source technologies and frameworks you can use:

LLMs

Llama 3.x, Qwen 3, DeepSeek

Embeddings

BGE-M3, e5, Snowflake Arctic

Vector DBs

Qdrant, Milvus, pgvector, Chroma

Serving

Ollama, vLLM

Frameworks

LangGraph, LlamaIndex, Haystack

Model Libraries

Hugging Face Transformers, Sentence Transformers

Validation

Pydantic AI, Instructor

Rerankers

BGE Reranker, Jina Reranker, Cohere Rerank (API)